AI-Assisted Covenant Monitoring

A Working Prototype

Project Documentation and Technical Reference

Prepared by Aadhitya

August 2026

Built as an independent technical exploration ahead of an interview for the Junior Portfolio Manager role at Enpal. Not an Enpal-commissioned project.

Confidentiality Notice. This project, and this document, are demonstrated entirely on synthetic, self-authored loan agreements and synthetic portfolio data. No real Enpal transaction documents, and no real document belonging to any other institution, are used anywhere in this project. In a production setting, agreement text and portfolio data would be processed entirely within a private environment. Nothing described here reflects, or is derived from, any real deal document.

Executive Summary

This document describes a working prototype built to explore a specific idea: that artificial intelligence, applied carefully and with a human always in the loop, can meaningfully help a small portfolio team track the financial covenants attached to their lending facilities – even when there are only a handful of such facilities, not hundreds.

A natural objection follows immediately: if there are only a handful of facilities, why does any of this need a system at all? It is a fair question, and this document takes it seriously rather than assuming it away. The answer does not rest on document volume. It rests on three things that are real regardless of how many facilities exist: a covenant is tested every reporting period, for years, not read once; two lenders can use the identical covenant name while meaning different things by it; and an amended covenant requires knowing which version currently governs, as of a specific date – not just what the original agreement once said.

To test whether these three problems are real, and whether a carefully designed system can actually address them, I built three entirely fictional loan agreements, wrote a small pipeline of five Jupyter notebooks around them, and ran the whole thing end to end. Every number in this document is a real, computed result from that pipeline – not a projection of what it should do.

The headline results: the system correctly told apart two facilities that used an identical covenant name but meant different things by it, with the difference actually changing the risk conclusion. It correctly resolved which threshold governed a covenant across time after that covenant was formally amended, including a case where a naive system – one that only read the original agreement and never learned of the amendment – would have wrongly concluded that two genuinely compliant periods were covenant breaches. And where the evidence was not clean enough to trust automatically, the system said so, and withheld a result rather than guess.

This remains a rough, evolving prototype, built in my own time on a small and deliberately limited set of synthetic documents. It is not a finished product, and it is not what Enpal would need in production. But it is real, it runs, and every claim in it can be checked against the code and the data behind it.

Contents

1 Introduction and Premise

1.1 The Context: Enpal's Financing Model

Enpal finances the sale of residential solar systems, battery storage, and related equipment to households, and then refinances that growing pool of customer contracts through structured financing arrangements with institutional lenders. This is a capital-efficient model: rather than holding all of that lending on its own balance sheet indefinitely, Enpal sells or pledges pools of customer receivables into special-purpose financing vehicles funded by banks and asset managers.

Over the past few years, this financing base has grown steadily. A first warehouse facility was established in 2023 with Citi and M&G. Within a year, two further asset-backed warehousing facilities followed, backed by Barclays Europe, Bank of America, and Crédit Agricole CIB, alongside mezzanine financing from CPP Investments. In 2024, the European Investment Bank Group backed what has been described as Europe's first public solar securitisation. By 2025, a further warehouse facility of several hundred million euros had been arranged with M&G. Each of these facilities is a real, separately negotiated legal agreement, typically containing financial covenants – contractual promises about the health of the underlying loan pool or the company's own balance sheet – that must be tested and reported on every period, for as long as the facility remains outstanding.

1.2 Why This Is Worth Solving, Even With Few Facilities

A natural objection: Enpal has roughly half a dozen distinct facility agreements, not hundreds. A handful of documents is not, on its own, a volume problem a small team cannot handle by hand – so why does this need a system at all?

The case rests on three separate observations, each of which is real even with a small number of facilities:

1. **Recurrence.** A covenant is not read once and filed away. It is tested every single reporting period – monthly or quarterly, depending on the facility – for the entire multi-year life of the agreement. The risk is not in the first read; it is in staying accurate across dozens of repeated testing cycles, some of which may happen years apart.
2. **Definitional inconsistency.** Two different lenders can use the exact same covenant name – a delinquency ratio, a leverage ratio – while defining it in materially different ways: a different threshold, a different denominator, a different aging bucket. This risk exists whether there are three facilities or three hundred; a person can misapply one lender's convention to another's data just as easily either way.
3. **Amendment and lifecycle drift.** Loan agreements are not static. They get amended, waived, and renegotiated over their life. Knowing which version of a covenant is *currently* governing, as of a specific date, is a genuine and recurring problem – not something solved by reading the original document once.

1.3 What This Project Sets Out to Prove

Rather than argue these points in the abstract, this project builds a small, self-contained system and tests whether it actually handles all three problems correctly, on real (if synthetic) documents and real (if synthetic) financial data. Concretely, the goal was to demonstrate:

- Extraction of covenant terms from unstructured legal text, with every extracted value grounded in an exact quotation from the source document – never a guess, and never invented.
- A genuine measure of confidence in that extraction, based on whether independent, repeated attempts agree with each other – not on the model's own self-reported certainty.
- Correct identification of definitional inconsistency between two facilities that happen to use the same

covenant name.

- Correct resolution of which version of an amended covenant governs, as of any given date.
- A working comparison layer that takes real portfolio data and produces an actual period-by-period compliance and early-warning result.

2 Design Choices and Scope

2.1 Why Synthetic Data

Every document and every figure used anywhere in this project was written specifically for this project. No real Enpal transaction document was used, and no real document belonging to any other institution was used either, even though such documents are often legally public. Several reasons drove this choice, and they reinforced each other:

Consistency of principle. The entire premise of this project is that sensitive financing documents should never leave a private environment. Publishing even someone else's real, legally public credit agreement in a demonstration repository would contradict that principle in spirit, even if not in law. Better to hold the standard consistently than to make an exception for convenience.

Precision of design. Self-authored documents can be built to demonstrate exactly the right thing. A real filed agreement was not written with this project's three specific claims in mind, and finding one that cleanly isolates definitional inconsistency, or a clean amendment cutover, would be more luck than engineering.

Accountability. If asked about any specific clause, the honest answer is always available, because every clause exists because it was written to demonstrate something specific.

2.2 The Three Synthetic Facilities

Three fictional financing facilities were written. Each isolates exactly one of the three problems described in Section 1.2, so that each claim could be tested cleanly, without one facility's complications interfering with another's. None of the three was drafted arbitrarily – each design choice below, including the specific covenant register, the specific definitional variation, and the specific amendment structure, was made for a stated reason.

Facility A – the control. Structured as an asset-backed warehouse facility, matching Enpal's real, primary financing model. It carries two covenants – a Delinquency Ratio and an Overcollateralization Test – chosen because they represent the two fundamental covenant families present in almost any securitisation or warehouse facility: a pool-performance trigger, and a collateral-sufficiency test. Both were drafted with reference to real, publicly filed asset-backed servicing agreements and investor reports, so that the drafting conventions – how a delinquency trigger is defined, how an overcollateralization target is calculated – reflect genuine market practice rather than an invented approximation of it. Facility A is deliberately uneventful: every period stays comfortably within its limits. This matters for two reasons. First, if the extraction and testing pipeline cannot handle the easy, healthy case cleanly, nothing else in the project is worth trusting. Second, Facility A serves as the reference point against which Facility B's entire argument is measured – Facility B reuses Facility A's exact covenant name, on purpose.

Facility B – definitional inconsistency, isolated. Also structured as an asset-backed warehouse facility, with a different lender, so that the only meaningful difference between it and Facility A is the covenant definition itself, not the surrounding facility type. Facility B's Delinquency Ratio carries the identical name as Facility A's – "Delinquency Ratio" – but a materially different governing definition: receivables 90 or more days delinquent, divided by the pool's *fixed original* balance at closing, rather than Facility A's 60-or-more-days delinquent, divided by the pool's *current*, amortising balance. This specific combination of differences – both the delinquency threshold and the denominator – was not invented arbitrarily; it mirrors a real, observed variation across actual filed asset-backed servicing agreements, where both conventions are genuinely in use. The point of Facility B is to test whether identical labels concealing different meanings is a real, detectable risk, and the definitional difference was chosen specifically because it is subtle enough that a person moving quickly between two facilities could plausibly miss it. Facility B also carries a Reserve Account covenant, unrelated to the delinquency story, included so that Facility B reads as a genuine, independent facility in its own right rather than merely "Facility A with different

numbers," and so that the confidence-review mechanism (Section 5) would have a second, unrelated covenant to test.

Facility C – amendment and lifecycle drift, isolated. Deliberately written in a different register entirely: a corporate term loan, in the leveraged-finance style, rather than an asset-backed structure. This switch serves two purposes. First, it mirrors Enpal's actual financing stack, which mixes asset-backed warehouse facilities with corporate-level borrowing, so the project demonstrates the approach generalising across registers rather than working only within one. Second, it keeps the amendment-resolution problem cleanly isolated: Facility C's single covenant, a Consolidated Net Leverage Ratio, is the most widely recognised covenant type in corporate lending, chosen specifically so that the complexity being demonstrated is the amendment mechanic itself, not an unfamiliar covenant type. The original agreement includes a limited equity cure right, a genuine and common market feature in leveraged facilities, included here so that the later amendment has a believable backstory: a cure was already used once, making a formal amendment the natural next step rather than the first resort. The amendment itself follows the real structural pattern observed in actual filed credit agreement amendments: a recital acknowledging an anticipated covenant breach, a waiver of that specific default, and a restated covenant carrying a new, tiered threshold with its own effective date. The tiering was deliberately set so that the amendment's effective date falls on a fiscal quarter boundary, and so that the change is tested against a period the resolver must key off the correct *test date*, not the amendment document's own signing date – the single hardest and most realistic version of this problem.

Facility C additionally carries a second document: the formal amendment described above, executed after the original agreement. This is the document that makes the amendment-resolution claim testable at all.

Facility	Register	Isolates	Key covenants
A	Asset-backed warehouse	Control case – no complications	Delinquency Ratio (60+ days / current pool balance), Overcollateralization Test
B	Asset-backed warehouse	Definitional inconsistency	Delinquency Ratio (90+ days / fixed original pool balance) – same name as Facility A, different meaning; Reserve Account covenant
C	Corporate term loan	Amendment and lifecycle drift	Consolidated Net Leverage Ratio, later formally amended

2.3 Real-World Grounding

To make the synthetic documents read like genuine legal drafting rather than an invented approximation of it, several real, publicly filed documents were consulted purely as structural and stylistic references – never copied, and never fed into the system itself. These included real asset-backed servicing agreements and investor reports filed with the U.S. Securities and Exchange Commission, and several real credit agreement amendments. Where a specific facility's design drew directly on one of these references, that is noted in Section 2.2 above rather than repeated here.

3 System Architecture

3.1 Pipeline Overview

The system is built as five sequential Jupyter notebooks, each producing a saved output that the next notebook reads and builds on. Each notebook is deliberately self-contained – able to be read, run, and understood on its own – rather than relying on shared code imported from elsewhere.

#	Notebook	What it does
1	Extraction	Reads each synthetic agreement and extracts covenant terms, with every value grounded in an exact quotation from the source text.
2	Confidence & Review	Repeats extraction three times per document and checks whether the results agree, combined with a structural validation check. Disagreement is flagged for human review rather than silently accepted.
3	Record Store	Builds a versioned record of every approved covenant, and resolves which version – original or amended – governs as of any given date.
4	Comparison & Alerting	Compares real portfolio and financial data against the resolved governing threshold, period by period, producing an actual compliance and early-warning result.
5	Visualizations	Builds the two charts included in this document, directly from the saved results of Notebook 4.

3.2 Technology Stack

The extraction step (Notebook 1) makes real calls to Anthropic's Claude API (model: Claude Sonnet 5), using a schema-constrained tool call so that the model's response is forced into a fixed structure rather than free-form text. Notebooks 2 through 5 involve no external API calls at all – they are ordinary, deterministic Python, using `pandas` for data handling, `pydantic` for structural validation, `python-dateutil` for date parsing, and `matplotlib` for the charts.

4 Notebook 1: Extraction

4.1 What It Does

This notebook takes each of the four synthetic documents – Facility A's agreement, Facility B's agreement, Facility C's original agreement, and Facility C's amendment – and asks the model to identify every financial covenant it contains, returning a structured record for each one.

4.2 How It Works

The request to the model is built so that it can only respond in one specific structured shape: a list of covenants, each with a name, a governing definition, a threshold value, and – critically – an exact quotation from the source document supporting each of those fields. If a value cannot be supported by an exact quotation, the model is instructed to say so explicitly rather than infer or guess.

This matters because it is the mechanism behind the central anti-hallucination claim of the whole project: the model is not being asked to summarize or interpret the document in its own words. It is being asked to locate and cite specific spans of real text. After the model responds, a separate, independent check – ordinary Python code, not another model call – verifies that every quotation the model returned is genuinely, character-for-character, present in the source document. This is a real, mechanical check, not something the model reports about itself.

4.3 Results

Across the four documents, six distinct covenants were correctly identified, each with a verified citation:

Document	**Covenant**	**Threshold**	**Citation**
Facility A	Delinquency Ratio / Trigger	5.00%	Verified
Facility A	Overcollateralization Test	8.00%	Verified
Facility B	Delinquency Ratio / Trigger	4.00%	Verified
Facility B	Reserve Account Covenant	2.00%	Verified
Facility C (original)	Consolidated Net Leverage Ratio	3.50:1.00	Verified
Facility C (amendment)	Consolidated Net Leverage Ratio	3.50:1.00 then 4.00:1.00	Verified (both tiers)

Worth noting explicitly: the amendment's extraction correctly returned a *null* value for the covenant's underlying numerator-and-denominator definition, because the amendment document genuinely does not restate that definition – it only changes the threshold table. Rather than inventing a plausible-sounding definition to fill the gap, the system reported, accurately, that it could not be found in this particular document. This is exactly the behavior the anti-hallucination design is meant to produce.

5 Notebook 2: Confidence and Review

5.1 What It Does

Rather than trust a single extraction pass, this notebook runs the extraction process three independent times per document, and checks whether the three attempts agree with each other. This is combined with a separate structural check confirming that every extracted record has all the fields it is supposed to have.

5.2 The Method

Agreement is measured, not assumed. For each covenant, the notebook checks three things across the three independent passes: did every pass find the same underlying evidence (matched by the citation each pass quoted, since a citation is a verbatim quotation and should be stable if extraction is genuinely grounded); did every pass agree on the actual threshold value; and did every extracted record pass structural validation. A covenant is marked high confidence only if all three checks pass. Anything less is explicitly flagged for a human reviewer, rather than silently accepted.

5.3 A Real Finding: The Tier-Scoring Gap

While building this notebook, an early version of the scoring logic only checked the *first* threshold value belonging to each covenant. This went unnoticed at first because every covenant except one has only a single threshold. Facility C's amended covenant, however, has two – the pre-amendment threshold and the post-amendment threshold – and the early version was silently never checking the second one's agreement across passes at all. This was caught while building the next notebook, which needed to consume both thresholds independently, and was fixed by rewriting the scoring logic to check every threshold a covenant carries, not just the first. This is described in more detail in Section 9, but is worth flagging here as a genuine gap that was found and corrected, not a design that worked perfectly on the first attempt.

5.4 Results

Once the scoring logic was corrected to check every tier independently, seven distinct covenant-and-threshold combinations were scored across the four documents. Six reached full agreement across all three passes and were approved. One – Facility B's Reserve Account covenant – was correctly withheld.

> ***Key finding:*** The one item flagged for review was not a real disagreement about the underlying fact. All three passes quoted the identical source sentence. What differed was purely how the threshold value was phrased: one pass wrote "2.00% of the Original Pool Balance," the other two wrote the shorter "2.00%." Same fact, two valid phrasings, correctly caught as an inconsistency worth a person's attention – and resolvable by a person in a few seconds. This is precisely what a review queue is for.

6 Notebook 3: Record Store

6.1 What It Does

This notebook takes every covenant that passed Notebook 2's confidence check, and builds a single, queryable record answering one question: as of any given date, what is the currently governing version of this covenant?

6.2 The Amendment Problem, Explained Simply

Facility C's original agreement, read on its own, states a single leverage ratio limit of 3.50:1.00, with no end date – as far as that document alone is concerned, that limit applies forever. Later, a formal amendment changes the limit to 4.00:1.00, but only from a certain date onward, and only after confirming the original 3.50:1.00 limit applied up to that point. A system that only reads the original document would never learn about this change. A system that reads both documents, but does not know which one to trust for which dates, risks getting the answer wrong in either direction.

6.3 How Resolution Works

Each covenant record carries the date range it governs. Where an amendment exists, its date ranges take priority over the original document's, for any date the amendment actually covers; the original document's terms only matter for dates the amendment does not address. In practice, for Facility C, this means: for any test date up to the end of 2024, the governing threshold is 3.50:1.00; for any test date from March 2025 onward, it is 4.00:1.00 – correctly picked up from the amendment, not the original document.

6.4 Results

> ***Key finding:*** Resolving a June 2025 test date correctly returns the amended 4.00:1.00 threshold. A deliberately naive version of the same resolver – one built to only ever consult the original agreement – returns 3.50:1.00 for the same date instead. This is not a hypothetical concern: it is the exact, demonstrated difference between a system that correctly tracks amendments and one that does not.

Facility B's Reserve Account covenant, still pending human review from Notebook 2, is deliberately and correctly absent from this record store. An unresolved covenant is unavailable to the resolver, not silently included as though it had been approved.

7 Notebook 4: Comparison and Alerting

7.1 What It Does

This is the notebook where everything comes together: real (synthetic) portfolio and financial data, spanning several consecutive reporting periods per facility, is compared against the correctly resolved governing threshold for each period, producing an actual, period-by-period compliance result.

Each facility's raw underlying data is shown immediately before its computed results below, limited in each case to only the figures that actually feed into that facility's reported covenant math – so the arithmetic behind every ratio in this section can be checked directly, rather than taken on faith.

7.2 Facility A – The Control Case

Raw portfolio data. All monetary figures in EUR millions.

Period	Pool Balance	Advance Balance	61–90 Days	91–120 Days	120+ Days
2024-03	150.00	132.00	0.54	0.42	0.24
2024-04	156.00	137.28	0.65	0.50	0.30
2024-05	161.00	141.68	0.76	0.59	0.35
2024-06	164.00	144.32	0.85	0.66	0.39
2024-07	167.00	146.96	0.94	0.73	0.43
2024-08	169.00	148.72	1.03	0.80	0.47
2024-09	171.00	150.48	1.12	0.87	0.51
2024-10	172.50	151.80	1.19	0.92	0.54

The Delinquency Ratio each period is the sum of the 61–90, 91–120, and 120+ day balances, divided by the Pool Balance. The Overcollateralization Amount is the Pool Balance minus the Advance Balance, compared against a target of 8.00% of the Pool Balance.

Computed results. Delinquency Ratio and Overcollateralization Test were computed across eight monthly reporting periods. Both stayed comfortably within their respective limits throughout, exactly as intended for a clean control case.

Period	Delinquency Ratio	vs. 5.00% Trigger	Overcollateralization
2024-03	0.800%	OK	Satisfied (12.0% vs. 8.0% target)
2024-04	0.929%	OK	Satisfied
2024-05	1.056%	OK	Satisfied
2024-06	1.159%	OK	Satisfied
2024-07	1.257%	OK	Satisfied
2024-08	1.361%	OK	Satisfied
2024-09	1.462%	OK	Satisfied
2024-10	1.536%	OK	Satisfied

7.3 Facility B – Definitional Inconsistency

Raw portfolio data. All monetary figures in EUR millions. The Original Pool Balance is fixed at closing and does not change period to period; the current Pool Balance amortises downward as the underlying receivables are repaid.

Period	Original Pool Bal.	Current Pool Bal.	61–90 Days	91–120 Days	120+ Days
2024-11	220.00	218.00	0.90	0.40	0.20
2024-12	220.00	211.00	1.40	0.65	0.32
2025-01	220.00	204.00	2.20	1.05	0.52
2025-02	220.00	197.00	3.40	1.70	0.85
2025-03	220.00	190.00	5.20	2.70	1.35
2025-04	220.00	183.00	7.80	4.10	2.05

Facility B's own, correct Delinquency Ratio is the sum of the 91–120 and 120+ day balances (its 90-or-more-days definition), divided by the fixed Original Pool Balance. The "if the wrong formula were used" column below instead sums the 61–90, 91–120, and 120+ day balances (a 60-or-more-days definition), divided by the current, amortising Pool Balance – using the identical raw data, computed with the wrong calculation convention.

Computed results. The same underlying raw delinquency data was computed two different ways: correctly, using Facility B's own contractual definition, and incorrectly, using a different calculation convention instead – the realistic failure of reusing a familiar calculation template across facilities without checking the governing terms of the specific facility in front of them. Both results are tested throughout against Facility B's own real, contractual trigger.

Period	B's own definition	vs. 4.00% trigger	If wrong formula used	vs. B's own 4.00% trigger
2024-11	0.273%	OK	0.688%	OK
2024-12	0.441%	OK	1.123%	OK
2025-01	0.714%	OK	1.848%	OK
2025-02	1.159%	OK	3.020%	OK
2025-03	1.841%	OK	4.868%	**FALSE BREACH**
2025-04	2.795%	OK	7.623%	**FALSE BREACH**

Key finding: Under its correct, contractual definition, Facility B never comes close to breaching its 4.00% trigger. Using the wrong calculation formula instead, on the identical underlying data, produces a genuine false breach against Facility B's own real threshold, one period earlier than the correct calculation ever comes close. No threshold number was mixed up here – the wrong calculation convention was simply applied to this facility's data instead of its own.

7.4 Facility C – Amendment Resolution

Raw financial data. All monetary figures in EUR millions.

Test Period End	Net Debt	EBITDA (LTM)	Cure Contribution
2023-09-30	178.00	64.50	–
2023-12-31	184.00	63.50	–
2024-03-31	190.00	62.00	–
2024-06-30	196.00	60.50	–
2024-09-30	202.00	59.00	–
2024-12-31	209.00	57.00	3.00
2025-03-31	215.00	57.80	–
2025-06-30	213.00	59.00	–

The Consolidated Net Leverage Ratio each period is Net Debt divided by EBITDA; where a Cure

Contribution was applied, it is added to EBITDA for that period's calculation only, per the original agreement's cure mechanic.

Computed results. The Consolidated Net Leverage Ratio was computed across eight quarterly test periods, each compared against whichever threshold was actually governing on that date.

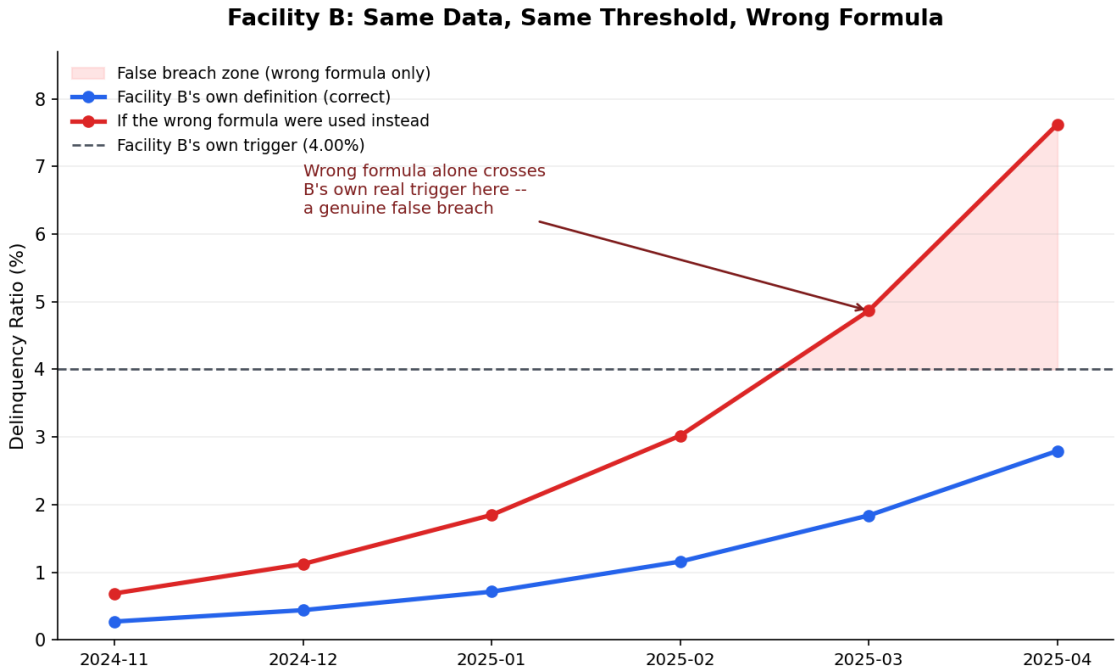
Test Period	Ratio	Governing Threshold	Result	Headroom
2023-09-30	2.760x	3.50x	Compliant	+0.740x
2023-12-31	2.898x	3.50x	Compliant	+0.602x
2024-03-31	3.065x	3.50x	Compliant	+0.435x
2024-06-30	3.240x	3.50x	Compliant	+0.260x
2024-09-30	3.424x	3.50x	Compliant	+0.076x
2024-12-31	3.483x*	3.50x	Compliant	+0.017x
2025-03-31	3.720x	**4.00x**	Compliant	+0.280x
2025-06-30	3.610x	**4.00x**	Compliant	+0.390x

*Adjusted for the permitted equity cure of EUR 3.00 million shown above; the unadjusted ratio this period was 3.667x, which would have breached the 3.50x limit absent the cure.

Key finding: Every period is genuinely compliant. But a naive system – one that never learned of the amendment and kept testing every period against the original 3.50x limit – would have wrongly concluded that both the March 2025 and June 2025 periods were covenant breaches (3.720x and 3.610x both exceed 3.50x). Both are, in fact, comfortably compliant under the correctly resolved 4.00x threshold that actually governs those dates. This is the single sharpest, most concrete result in the entire project.

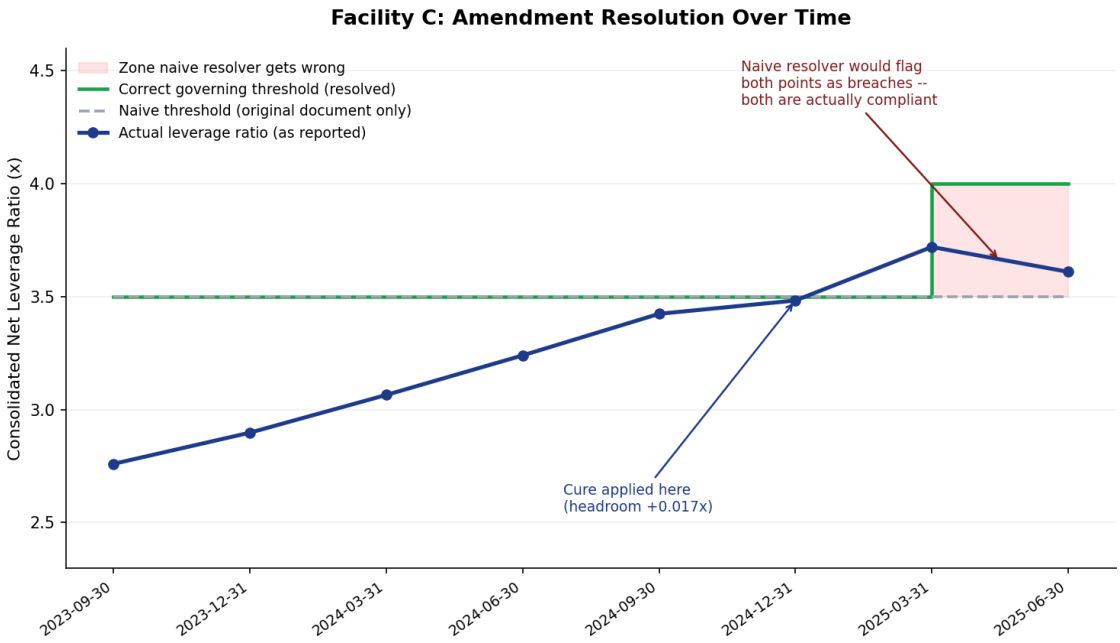
8 Visualizations

8.1 Chart 1: Facility B – Same Data, Same Threshold, Wrong Formula



This chart makes the definitional-inconsistency finding visible at a glance. The blue line – Facility B's correct, contractual definition – stays low and never threatens its own 4.00% trigger. The red line – the same underlying data, run through the wrong calculation formula instead – climbs steeply and crosses Facility B's own real trigger by March 2025, shaded here as the false-breach zone. Same facts, same threshold, one wrong formula, two entirely different conclusions about portfolio risk.

8.2 Chart 2: Facility C – Amendment Resolution Over Time



This chart shows the leverage ratio's real trajectory (dark blue) against the correctly resolved governing

threshold (green), which visibly steps up from 3.50x to 4.00x at the amendment's effective date. The narrow point in late 2024, where the line nearly touches the threshold, is the period the equity cure was needed. The shaded region marks where a naive, amendment-unaware system (grey dashed line) would have wrongly concluded a breach, even though both of those periods are genuinely compliant under the correct, amended threshold.

9 Engineering Discipline: What Went Wrong, and How It Was Fixed

This project's central argument is that a system like this must verify rather than blindly trust – both the covenant data it extracts, and its own internal reasoning. In the spirit of that same discipline, this section documents the real problems encountered while building it, rather than presenting a version of events where everything worked correctly the first time. None of these were catastrophic, and all were caught through deliberate testing before they could produce a wrong result – which is, itself, the point.

- **Inconsistent API response shapes.** On different calls, the extraction model occasionally returned its structured list of covenants wrapped differently – sometimes as a plain list as expected, sometimes as that same list encoded as a text string, and once as a further nested object containing the list. Each of these was caught through direct inspection of the raw response before being allowed to propagate further, and the parsing logic was made robust to all of them at once, rather than patched separately each time.
- **Overly strict citation matching.** An early version of the multi-pass agreement check in Notebook 2 required two citations to match exactly, character for character, to be considered the same underlying evidence. In practice, the model occasionally quoted slightly different boundaries of the same real sentence across different passes – for instance, including or omitting a leading phrase. This produced false disagreement flags on genuinely consistent extractions. The fix was to treat two citations as the same evidence if one is fully contained within the other, which was verified against the real failing case before being adopted.
- **A silent date-parsing error.** A general-purpose date parser, used to interpret extracted phrases like "Closing Date (March 14, 2024)," was found to silently misread the year in some cases – returning 2026 instead of 2024 – because a closing parenthesis immediately following the year confused the parser's handling of that token, causing it to quietly substitute the current year instead of raising an error. This was caught only because every date string was deliberately tested against its expected value before being trusted; the fix was to extract and parse any parenthetical date content in isolation first.
- **The tier-scoring gap** described in Section 5.3, where only the first of two threshold values on an amended covenant was being independently checked for cross-pass agreement.
- **A data-type serialization issue.** Values computed using the `pandas` data library are not, by default, compatible with the standard method used to save results as a text file, and would have caused the final save step to fail. This was caught by testing the save step in isolation before relying on it, and fixed with a small conversion step.

10 Key Findings Summary

Claim	Evidence
Extraction can be grounded, not generative	Every extracted value carries an independently verified, exact quotation from source text; where no definition existed in a document, the system correctly reported that, rather than inventing one.
Confidence can be genuinely measured	Across three independent extraction passes and seven covenant-threshold combinations, six reached full agreement and were approved; one was correctly withheld, with a specific, human-understandable reason.
Definitional inconsistency is real and detectable	The identical covenant name, across two facilities, produces materially different – and differently conclusive – results when tested against the same underlying data.
Amendments can be correctly resolved by date	A naive system would have wrongly flagged two genuinely compliant periods as covenant breaches after Facility C's amendment took effect. The system built here did not.

11 Limitations

- **Synthetic data only.** Every agreement and every financial figure used in this project was written specifically for it. Real-world drafting conventions were used only as structural and stylistic reference, never copied.
- **Starts after document scanning.** Agreements are provided as plain text throughout this project. A production system would need a preceding step to convert scanned or image-based documents into text; that step is out of scope here.
- **A deliberately small number of facilities.** Three facilities is a design choice made to isolate three specific problems cleanly, not a limitation to be apologized for – see Section 1.2 and Section 2.2.
- **A rough, evolving prototype.** This is not a finished product, and was never intended to be one at this stage. It is a genuine, working first attempt at a real problem.

12 Conclusion and Way Forward

This project set out to test whether a carefully designed system – one that extracts covenant terms with real citation grounding, measures its own confidence honestly rather than asserting it, and correctly resolves which version of a covenant governs over time – could meaningfully help with a real, if currently small-scale, operational problem. On the evidence produced here, it can.

Just as importantly, building it surfaced genuine engineering problems along the way – in the extraction mechanism, in the confidence-scoring logic, and in ordinary date-handling code – each of which was caught through deliberate testing rather than assumed away. That discipline, applied to the system itself and not only to the covenants it reads, is, I think, the more durable lesson of this project: augmenting judgment means nothing if the tool doing the augmenting has not itself been checked.

The natural next steps, were this to continue: independently confidence-scoring every threshold tier from the start rather than discovering the gap partway through; extending the synthetic document set to cover a wider range of covenant types; and eventually testing the same approach against a genuinely messy, scanned real-world document, once the underlying extraction logic has proven itself on clean text.